



BIRC 商智中心

Business Intelligence Center — 推動校園數位人才培育



全國競賽獲獎



業師輔導



專案實作

2026 後端群體驗營

```
$ java OrderSystem --mode interactive
```

Backend 體驗營 Day2

// 訂單系統 — 從陣列到「真實世界的後端」

平行陣列
組織多筆資料

庫存判斷
避免超賣的關鍵

訂單邏輯
購物車的靈魂



昨天回顧

Day 1 學了什麼

- ✓ 變數 — 存東西的盒子
- ✓ 條件判斷 — if/else 分支
- ✓ 迴圈 — while 重複執行
- ✓ Scanner — 讀取輸入



今天要做的事

-  用平行陣列存商品資料
-  做一個超簡化的購物車
-  加入庫存判斷（避免超賣）



重點：同樣的工具（變數、if/else、while、陣列），可以做出完全不同的東西！



今日目標



做出一個可互動的訂單系統

商品清單 + 選購 + 結帳



理解「庫存扣減」的邏輯

買到剩 0 件就不能再買



看見「陣列 → 資料庫」的關聯

購物車



購物車的靈魂

你點外送 App 按下「加入購物車」後，後端發生了什麼？

1. 讀取商品

從資料庫拿商品清單



2. 檢查庫存

還有貨嗎？



3. 扣庫存

把它從架上拿掉



4. 計算金額

加總所有商品



這些都是後端 — 你看不到，但它一直在運作



步驟一：商品清單

Phase 1 — 用平行陣列存商品資料

```
1 // 平行陣列：同 index 對應同一個商品
2 String[] products = {"cola", "sandwich", "chips", "cookie", "juice"};
3 int[] prices = {30, 65, 45, 35, 40};
4
5 // 印出商品選單
6 for (int i = 0; i < products.length; i++) {
7     System.out.println((i + 1) + ". " + products[i] + " → " + prices[i] + " 元");
8 }
9
10 // 讓使用者選購
11 System.out.print("請輸入想買的商品編號 (1-" + products.length + ") : ");
12 int choice = scanner.nextInt();
```



平行陣列：兩個陣列長度相同，同一個 index 對應同一個商品

1 2
3 4

陣列索引 Index 是什麼？

Step 1.5 — 陣列的「門牌號碼」

```
1 // 陣列的 index 從 0 開始！
2 String[] products = {"cola", "sandwich", "chips", "cookie", "juice"};
3
4 //           ↑       ↑       ↑       ↑       ↑
5 //   index:      0       1       2       3       4
6 // products[0] = "cola"
7 // products[1] = "sandwich"
8 // products[4] = "juice"    ← 最後一個是 length - 1
```

⚠ 常見錯誤

products[5] →

ArrayIndexOutOfBoundsException

index 最大只能是 length - 1

💡 為什麼從 0 開始？

index 代表「距離起點幾個位置」

第 1 個元素 = 距離起點 0 個位置



你剛剛用了什麼？



平行陣列

products + prices

同一 index = 同一商品



For 迴圈

```
for (int i = 0; ...)
```

印出所有商品



條件判斷

```
if (choice >= 1 && ...)
```

確認輸入範圍



重點：你昨天學的工具，今天馬上用上了！



步驟二：多商品訂單

Phase 2 — 可以買多個商品，計算總金額

```
1  int totalAmount = 0; // 總金額
2  int orderCount = 0;  // 買了幾個
3  while (true) {
4      System.out.println("商品列表：");
5      for (int i = 0; i < products.length; i++) {
6          System.out.println((i + 1) + ". " + products[i] + " → " + prices[i] + " 元");
7      }
8      System.out.println("0. 結帳");
9      System.out.print("請選擇商品編號：");
10     int choice = scanner.nextInt();
11
12     if (choice == 0) {
13         break; // 結帳跳出迴圈
14     }
15
16     if (choice >= 1 && choice <= products.length) {
17         System.out.print("買幾個？");
18         int quantity = scanner.nextInt();
19         int subtotal = prices[choice - 1] * quantity;
20         totalAmount += subtotal;
21         orderCount += quantity;
22     }
23 }
```



迴圈的妙用



while (true) 迴圈

- • 不斷顯示商品選單
- • 讓使用者可以一直買
- • 輸入 0 才跳出

→ 這就是「購物車」的迴圈！



累加金額

- • `totalAmount += subtotal`
- • 每買一個就加一次
- • 最後就是總金額

→ 這就是「結帳」的計算！



重點：迴圈讓使用者可以一直買，`break` 讓使用者可以結帳



步驟三：庫存判斷（重點）

Phase 3 — 買到剩 0 件就不能再買

```
1  int[] stock = {10, 5, 8, 12, 7}; // 庫存數量
2
3  // 檢查庫存
4  if (stock[index] == 0) {
5      System.out.println("❌ " + products[index] + " 已售完，無法購買！");
6      continue; // 跳過這次，回到迴圈開頭
7  }
8  // 庫存不足
9  if (quantity > stock[index]) {
10     System.out.println("❌ 庫存不足！只剩 " + stock[index] + " 個");
11     continue;
12 }
13 // 扣庫存
14 stock[index] -= quantity;
```



陣列 → 資料庫



今天用的：平行陣列

```
String[] products = {...};  
int[] prices = {...};  
int[] stock = {...};
```

簡單、好懂、但有上限



真實世界：資料庫

```
SELECT * FROM products  
WHERE stock > 0  
ORDER BY price;
```

強大、可搜尋、可擴展



重點：你今天做的，就是資料庫在做的事的縮小版！



DomJudge 題庫

線上練習 — 挑戰自我

登入系統

網址：domjudge.ntubimdbirc.tw

帳號：學號

密碼：預設密碼（已發Email，如找不到請找助教）



題目說明

- ✓ 題目從簡單到困難
- ✓ 用 Java 作答
- ✓ 提交後系統自動批改
- ✓ WA 是答案錯誤 RE是程式執行編輯錯誤



操作說明影片



小提示：先完成遊戲再來挑戰，題目不會很難！



恭喜！

你今天做出了一個會動的訂單系統

這就是後端的真實面貌



範例程式碼

LucasHsu.dev — 2026 商智中心後端群體驗營